

ENTREGA DE AVANCES DEL SEMINARIO

FUNDAMENTOS DE SEGURIDAD

PRESENTADO A: TATIANA MARCELA GOMEZ SARMIENTO **INTEGRANTES:**

MATEO ELIECER MORENO TENJICA
JUAN CAMILO TORRES SUÁREZ
CRISTIAN SANTIAGO MORENO RUIZ
DAVID ALEJANDRO CONTRERAS DELGADO

19/09/2026

(Avance: incorpora diagrama de flujo del ataque y casos documentados)

1. Introducción

La incorporación de modelos de lenguaje de gran tamaño (LLM, por sus siglas en inglés) en aplicaciones de producción ha transformado la manera en que los sistemas de software generan y procesan información. Estos modelos ya no se limitan a responder preguntas dentro de una interfaz de chat: sus salidas se insertan en páginas web, se envían como comandos a otros sistemas, se ejecutan como fragmentos de código, se traducen en consultas a bases de datos o se utilizan como entradas para otros agentes de inteligencia artificial. Este cambio, de un rol pasivo a uno activo dentro de la arquitectura de una aplicación, amplía de forma significativa la superficie de ataque disponible para un adversario.

El presente informe desarrolla la vulnerabilidad conocida como Manejo Inseguro de Salidas del LLM (Insecure Output Handling / Improper Output Handling), catalogada dentro del proyecto OWASP Top 10 for LLM Applications. El objetivo central es mostrar, desde una perspectiva teórica y técnica, cómo una respuesta generada por un modelo de lenguaje puede convertirse en el vector de ataques como Cross-Site Scripting (XSS), inyección de consultas (SQL/NoSQL Injection) o inyección de comandos, cuando dicha salida se procesa o se muestra sin las validaciones adecuadas. A partir de este análisis, se plantean estrategias concretas de validación y codificación de salida que permiten mitigar el riesgo.

Este trabajo se enmarca dentro de un ejercicio comparativo más amplio sobre vulnerabilidades de agentes LLM (que incluye también autonomía excesiva, estado del arte de mecanismos de defensa y el diseño de una comparación experimental), por lo que el enfoque de esta sección se

centra específicamente en el punto 6 de la tabla de asignación de temas del curso: el manejo inseguro de salidas.

Respecto a la entrega inicial, este avance incorpora dos elementos nuevos solicitados: (i) un diagrama del flujo del ataque que ilustra visualmente el punto de decisión donde la aplicación debería validar la salida del LLM, y (ii) dos casos de incidentes reales y documentados (con CVE asignado) que evidencian que esta vulnerabilidad no es solo teórica, sino que ya ha sido explotada en librerías ampliamente usadas del ecosistema de agentes de IA.

2. Objetivos

2.1 Objetivo general

Analizar la vulnerabilidad de Manejo Inseguro de Salidas del LLM, explicando sus causas técnicas, sus vectores de explotación y las estrategias de mitigación aplicables, con el fin de demostrar cómo una respuesta generada por un modelo de lenguaje puede convertirse en una instrucción peligrosa para otros sistemas si no se valida y codifica adecuadamente.

2.2 Objetivos específicos

1. Describir el marco teórico y la ubicación de esta vulnerabilidad dentro del OWASP Top 10 for LLM Applications.
2. Explicar los mecanismos por los cuales una salida de LLM no validada puede derivar en ataques de tipo XSS, inyección de consultas o inyección de comandos.
3. Presentar un caso conceptual de explotación, apoyado en un diagrama de flujo, que ilustre el paso de datos entre el modelo, la aplicación y el usuario o sistema receptor.
4. Proponer e implementar un conjunto de controles de validación, sanitización y codificación de salida (output encoding) que mitiguen el riesgo identificado.
5. Evaluar el impacto de esta vulnerabilidad en la confidencialidad, integridad y disponibilidad de los sistemas que integran LLM, apoyándose en incidentes reales documentados.

3. Marco teórico

3.1 El OWASP Top 10 for LLM Applications

El proyecto OWASP GenAI Security desarrolla y mantiene el listado OWASP Top 10 for LLM Applications, una guía de referencia que identifica y clasifica los riesgos de seguridad más críticos en aplicaciones que integran modelos de lenguaje de gran tamaño. A diferencia del OWASP Top 10 clásico centrado en vulnerabilidades de aplicaciones web tradicionales, como inyección o configuración insegura, la iniciativa para LLM se enfoca en amenazas propias del ciclo de vida de estos modelos: manipulación de instrucciones (prompt injection), envenenamiento de datos y del

modelo, fuga de información sensible, riesgos de la cadena de suministro y manejo inadecuado de las salidas generadas.

En la edición 2025 de este listado, el manejo inseguro de salidas se identifica como LLM05: Improper Output Handling. Aunque descendió de la segunda a la quinta posición respecto a versiones anteriores, continúa considerándose un riesgo crítico, especialmente porque los LLM han pasado de responder preguntas de forma pasiva a enviar correos electrónicos, consultar bases de datos, invocar APIs y tomar decisiones de forma activa dentro de flujos de trabajo automatizados.

3.2 Definición de Manejo Inseguro de Salidas del LLM

El manejo inseguro de salidas ocurre cuando una aplicación confía en el contenido generado por un LLM y lo utiliza directamente para renderizarlo, ejecutarlo o reenviarlo a otro componente del sistema sin aplicar validación, escape (escaping) o sanitización adecuados. En esencia, el problema no reside en el modelo de lenguaje en sí, sino en la falta de un límite de confianza (trust boundary) claro entre lo que el modelo produce y lo que la aplicación consume.

La causa raíz es conceptualmente sencilla: un LLM es, desde el punto de vista de la seguridad de la aplicación, una fuente de datos no confiable, de la misma forma en que lo es la entrada de un formulario web o un parámetro de URL. Sin embargo, en la práctica muchas integraciones tratan la salida del modelo como si fuera contenido generado internamente por el propio sistema, y omiten los controles de sanitización que sí se aplican de forma rutinaria a otras fuentes externas.

3.3 Relación con otras vulnerabilidades del listado

El manejo inseguro de salidas no actúa de forma aislada: frecuentemente se combina con otras categorías del OWASP Top 10 for LLM Applications para materializar un ataque completo. La tabla siguiente resume las categorías más relacionadas.

Código	Categoría OWASP	Relación con el manejo inseguro de salidas
LLM01	Prompt Injection	Un atacante manipula la entrada para que el modelo genere una salida maliciosa que luego se procesa sin validar.
LLM05	Improper Output Handling	Vulnerabilidad central de este informe: ausencia de validación/sanitización de la respuesta del LLM.
LLM06	Sensitive Information Disclosure	La salida no filtrada puede exponer datos sensibles junto con contenido malicioso.

LLM08	Excessive Agency	Si la salida del LLM se ejecuta como comando con permisos amplios, el impacto de una salida maliciosa se amplifica.
-------	------------------	---

3.4 Estadísticas

Relevancia y Justificación Empírica: ¿Por qué debemos conocer y priorizar estos casos?

Para comprender la trascendencia del Manejo Inseguro de Salidas (OWASP LLM05: Improper Output Handling), es indispensable analizar el fenómeno a partir de métricas empíricas de la industria, mediciones de laboratorio en bancos de prueba (benchmarks) y evaluaciones de impacto financiero. Los incidentes documentados en librerías como Vanna.AI o LangChain no constituyen anomalías aisladas ni casos teóricos de laboratorio; representan la manifestación concreta de una brecha crítica y estructural entre el despliegue acelerado de modelos de lenguaje y la ausencia de controles deterministas sobre sus salidas.

A continuación, se presentan los datos cuantitativos que justifican la urgencia de priorizar esta vulnerabilidad en tres dimensiones:

1. Prevalencia de incidentes y brechas en entornos de producción

77% de organizaciones con brechas de seguridad en IA: De acuerdo con el AI Threat Landscape Report de la firma de seguridad HiddenLayer, el 77% de las compañías que integran modelos de lenguaje en sus flujos de trabajo reportó haber sufrido al menos un incidente o brecha de seguridad durante el último año.

88% de incidentes reportados en arquitecturas de agentes: El estudio especializado de Gravitee sobre la seguridad de agentes autónomos reveló que el 88% de las empresas experimentó incidentes de seguridad confirmados o sospechados originados por agentes de IA. En sectores altamente regulados como el sector salud, la cifra escala al 92.7%.

1 de cada 8 brechas empresariales ligada a agentes autónomos: Más del 12% de todas las brechas corporativas en entornos de inteligencia artificial ya involucran directamente a agentes autónomos que ejecutan herramientas, traducen consultas a bases de datos o manipulan código sin supervisión humana directa (HiddenLayer).

78 vulnerabilidades aisladas en familias de modelos líderes: En un estudio experimental conducido por Synapsed sobre cinco familias de modelos fundacionales (LLaMA, Gemma, Mistral, Qwen y Phi) evaluadas frente a la taxonomía OWASP GenAI, se documentaron 78 vulnerabilidades severas correspondientes exclusivamente al vector de Improper Output Handling (LLM05), demostrando que los modelos de lenguaje en sí mismos no pueden evitar generar respuestas con sintaxis explotable.

2. Tasas de éxito de explotación en bancos de prueba científicos (Benchmarks)

La comunidad científica y académica de ciberseguridad ha cuantificado con precisión la probabilidad de que una salida no validada desemboque en una acción perjudicial:

82.50% de tasa de acciones inseguras (NetInjectBench): En el estudio científico publicado bajo el benchmark NetInjectBench (arXiv:2607.10490), diseñado para evaluar agentes de administración de redes y ChatOps frente a inyecciones de instrucciones, la ejecución directa de la salida del modelo produjo una tasa de acciones no seguras (unsafe tool-action rate) del 82.5% en entornos donde no existía una capa intermedia de validación de salida.

24% a 40% de éxito en manipulación de agentes ReAct (InjecAgent - ACL 2024): El benchmark InjecAgent, presentado en la conferencia ACL 2024 (arXiv:2403.02691) y evaluado sobre más de 1,000 casos con arquitecturas comerciales como GPT-4, demostró que las inyecciones indirectas forzaron al agente a disparar llamadas a herramientas no autorizadas en el 24% de los casos, cifra que superó el 40% de éxito cuando las instrucciones del atacante fueron reforzadas con técnicas de evasión semántica (hacking prompts).

15% a 25% de código sintético con fallas de seguridad: Auditorías de AppSec demuestran que entre el 15% y el 25% del código generado automáticamente por LLMs contiene vulnerabilidades de seguridad comunes (como concatenaciones SQL sin sanear o código HTML desprotegido). Por consiguiente, cuando una aplicación reenvía este código a funciones dinámicas de ejecución (como el exec() en Vanna.AI), la probabilidad matemática de que se ejecute una instrucción destructiva es inminente.

3. Brecha de gobernanza, visibilidad técnica e impacto financiero

97% de brechas de IA sin controles de acceso: Según el Cost of a Data Breach Report de IBM Security, el 97% de las organizaciones que sufrieron una filtración de datos asociada a herramientas de IA carecían de políticas formales de control de acceso, segregación de privilegios y verificación de salidas para sus modelos.

Sobrecosto financiero de \$670,000 USD por Shadow AI: El mismo reporte de IBM demostró que la proliferación de herramientas de IA no monitoreadas ni validadas añade en promedio \$670,000 dólares en costos directos de remediación a una filtración de datos, elevando el costo medio global de una brecha a rangos entre \$4.44M y \$4.88M de dólares.

Solo el 21% de visibilidad real en directivos: A pesar del rápido despliegue de estas soluciones, únicamente el 21% de los directores de tecnología y seguridad corporativa (CISO/CTO) afirma tener visibilidad integral sobre los datos y sistemas del backend a los que tienen acceso los agentes de IA de su propia empresa (HiddenLayer).

Cuadro Resumen de Evidencia Empírica

Métrica / Indicador Analizado	Valor Cuantitativo	Fuente de Referencia	Implicación
Directa para LLM05			
Organizaciones con incidentes de IA en producción	77%	HiddenLayer AI Threat Landscape Report	Demuestra que la explotación de aplicaciones con LLM es un riesgo corporativo activo y cotidiano.
Incidentes de seguridad reportados en Agentes de IA	88% – 92.7%	Gravitee State of AI Agent Security	Las arquitecturas agénticas multiplican exponencialmente la criticidad de la salida generada.
Tasa de ejecución insegura de herramientas sin validación	82.5%	NetInjectBench (arXiv:2607.10490)	Si la aplicación confía ciegamente en la respuesta del modelo, el atacante domina la ejecución en 8 de cada 10 intentos.

Éxito de manipulación en agentes comerciales (GPT-4 / ReAct) 24% – 40% InjecAgent (ACL 2024) Es inviable delegar la seguridad en el criterio interno del modelo; se requieren controles deterministas externos.

Incidentes de IA ocurridos con ausencia total de controles 97% IBM Security Cost of a Data Breach Evidencia la carencia generalizada de límites de confianza (trust boundaries) en los despliegues de software modernos.

Sobrecosto financiero medio por IA no gobernada (Shadow AI) +\$670,000 USD IBM Security Cost of a Data Breach La falta de validación de salidas genera pérdidas económicas y regulatorias directas y cuantificables.

Referencias Bibliográficas Específicas de esta Sección (Normas APA 7)

Gravitee. (2026). The state of AI agent security report 2026. Gravitee.io Security Research.

<https://www.gravitee.io/blog/ai-agent-security>

HiddenLayer. (2024). Inaugural AI threat landscape report. HiddenLayer Security Research.

<https://hiddenlayer.com/research/>

HiddenLayer. (2026). 2026 AI threat landscape report: The rise of autonomous agent breaches.

HiddenLayer Research. <https://www.prnewswire.com/news-releases/hiddenlayers-inaugural-ai-threat-landscape-report-reveals-widespread-vulnerabilities-in-commercial-artificial-intelligence-302069796.html>

IBM Security. (2024). Cost of a data breach report 2024. IBM Corporation.

<https://www.ibm.com/reports/data-breach>

IBM Security. (2025). Cost of a data breach report 2025: Shadow AI and governance realities.

IBM Corporation. <https://www.ibm.com/reports/data-breach>

Shayoni, R. K., Shoaib, M. F., Hossain, S. M. A., & Mridha, M. F. (2026). NetInjectBench: Benchmarking indirect prompt injection in tool-using large language model agents for network operations. arXiv preprint arXiv:2607.10490. <https://arxiv.org/abs/2607.10490>

Synapsed. (2025). Empirical analysis of OWASP Top 10 vulnerabilities across foundation model families. Synapsed AI Security. <https://synapsed.ai/>

Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. Findings of the Association for Computational Linguistics: ACL 2024, 8740–8759. <https://aclanthology.org/2024.findings-acl.555/>

4. Análisis del problema: de la respuesta del LLM al ataque

4.1 El flujo de datos inseguro

Para comprender por qué una respuesta de un LLM puede convertirse en un vector de ataque, es necesario observar el flujo de datos típico de una aplicación integrada con un modelo de lenguaje:

1. Un usuario (potencialmente malicioso) envía una entrada a la aplicación, que se incorpora al prompt enviado al LLM.

2. El modelo genera una respuesta en lenguaje natural, código, marcado (HTML/Markdown) o una estructura de datos (JSON, SQL, comandos de shell).
3. La aplicación toma esa respuesta y la reenvía a un componente consumidor: el navegador del usuario (renderizado en el DOM), una base de datos, un intérprete de comandos, otra API o incluso otro agente de IA.
4. Si ese componente consumidor no aplica sus propios controles de sanitización, cualquier contenido malicioso presente en la salida del LLM —ya sea porque el modelo fue manipulado mediante prompt injection, porque reprodujo contenido de una fuente externa no confiable (por ejemplo, un documento o página web recuperados) o incluso por un comportamiento no intencionado del modelo— se ejecuta con los mismos privilegios que el resto del contenido legítimo.

El punto crítico de esta cadena es el paso 3: en ese momento la aplicación decide, implícita o explícitamente, si trata la salida del LLM como confiable o no confiable. El manejo inseguro de salidas ocurre precisamente cuando esa decisión se omite.

4.2 Diagrama del flujo del ataque

La Figura 1 sintetiza en forma gráfica la cadena descrita en la sección 4.1. El elemento clave del diagrama es la línea punteada que marca el límite de confianza (trust boundary): todo lo que ocurre por encima de esa línea (usuario, prompt, LLM y la decisión de la aplicación) determina si el contenido generado se trata como confiable o no; todo lo que ocurre por debajo (navegador, base de datos, shell/API) es donde el daño se materializa si el paso 3 no aplicó validación o codificación de salida. Las tres ramas inferiores corresponden exactamente a los tres vectores de ataque desarrollados en las secciones 4.3 a 4.5.

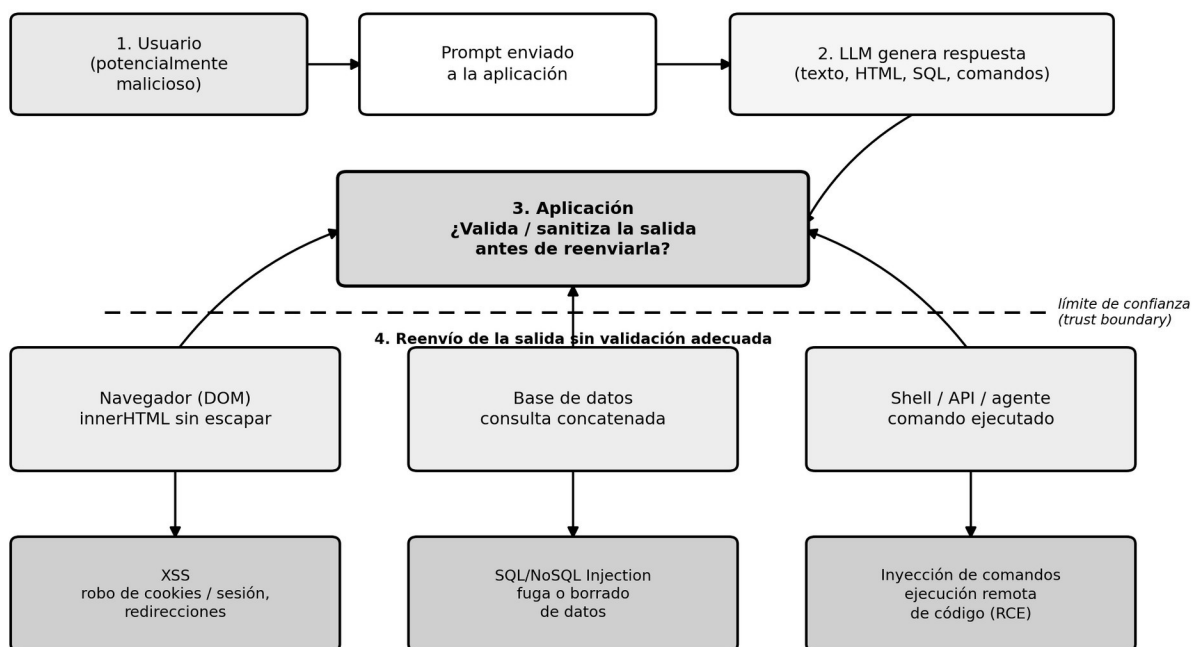


Figura 1. Flujo de datos inseguro desde el usuario hasta la explotación, mostrando el punto de decisión (paso 3) donde debería aplicarse validación / codificación de salida.

Figura 1. Flujo de datos inseguro desde el usuario hasta la explotación. Elaboración propia, basada en el modelo de flujo de OWASP LLM05:2025.

4.3 Vector de ataque 1: Cross-Site Scripting (XSS)

Cuando una aplicación web muestra la respuesta de un LLM directamente en el DOM —por ejemplo, insertándola mediante innerHTML o un componente equivalente que interpreta HTML— y esa respuesta contiene marcado o JavaScript, el navegador lo ejecutará como si fuera parte legítima de la página. Esto es análogo a un XSS almacenado o reflejado clásico, con la diferencia de que el vector de inyección no es un campo de formulario, sino el propio modelo de lenguaje.

Un LLM puede llegar a producir dicho contenido de varias formas: porque el atacante lo solicitó de manera directa disfrazando la petición como una tarea legítima (por ejemplo, pedir que se genere un fragmento de ejemplo en HTML), o porque una instrucción oculta en un documento procesado por el sistema (prompt injection indirecto) instruye al modelo para incluir dicho marcado en su respuesta.

Si la respuesta del modelo contuviera una etiqueta capaz de ejecutar script (por ejemplo, una etiqueta `<script>` o un manejador de evento embebido en una etiqueta HTML), el navegador la ejecutaría en el contexto de la sesión del usuario, permitiendo robo de cookies o tokens de sesión, redirecciones maliciosas o acciones no autorizadas en nombre de la víctima.

4.4 Vector de ataque 2: inyección de consultas (SQL / NoSQL Injection)

En arquitecturas donde un LLM traduce lenguaje natural a consultas de base de datos —un patrón habitual en asistentes de análisis de datos o agentes de "texto a SQL"— existe el riesgo de que la consulta generada por el modelo se ejecute directamente contra la base de datos sin pasar por una capa de validación o parametrización. Si el modelo es inducido, mediante manipulación del prompt o de los datos que procesa, a construir una consulta que incluya comandos destructivos o de exfiltración de datos, y la aplicación ejecuta dicha consulta de forma dinámica, el resultado es funcionalmente equivalente a una inyección SQL clásica, con el LLM actuando como intermediario involuntario.

4.5 Vector de ataque 3: inyección de comandos y ejecución de código

El riesgo se agrava en agentes de IA con capacidad de ejecutar acciones: llamar herramientas, ejecutar código generado dinámicamente, invocar APIs externas o interactuar con el sistema operativo. Si la salida del LLM se interpreta como un comando ejecutable sin una capa de validación de esquema, listas de permitidos (allow-lists) o confinamiento (sandboxing), un contenido malicioso en la respuesta del modelo puede traducirse en ejecución remota de código, modificación de archivos o llamadas no autorizadas a servicios externos. Esta situación se relaciona directamente con la categoría de Excessive Agency del OWASP Top 10 for LLM Applications: cuanto mayor sea la autonomía y los permisos concedidos al agente, mayor es el impacto potencial de una salida no validada.

5. Casos de incidentes documentados

Con el fin de sustentar que el manejo inseguro de salidas no es un riesgo únicamente teórico, esta sección presenta dos vulnerabilidades reales, con CVE (Common Vulnerabilities and Exposures) asignado, encontradas en librerías ampliamente utilizadas para construir aplicaciones y agentes basados en LLM. Ambos casos ilustran, con componentes distintos, el mismo patrón descrito en la Figura 1: una salida del modelo que cruza el límite de confianza sin la validación adecuada.

5.1 Caso 1 — CVE-2024-5565: ejecución remota de código en Vanna.AI

Vanna.AI es una librería de Python de tipo "texto a SQL" que permite a los usuarios consultar una base de datos en lenguaje natural; el modelo traduce la pregunta a una consulta SQL y, adicionalmente, ofrece una función de visualización de los resultados mediante la librería gráfica Plotly.

Investigadores de JFrog identificaron que la función `ask()`, cuando se ejecuta con la opción `visualize=True` (comportamiento por defecto), no solo genera la consulta SQL, sino que también le pide al LLM que genere el código Python necesario para graficar el resultado. Ese código generado por el modelo se ejecuta posteriormente con la función `exec()` de Python. Al no existir ninguna validación entre lo que el LLM genera y lo que `exec()` ejecuta, un atacante puede redactar una pregunta que, mediante prompt injection, induzca al modelo a incluir en su respuesta

comandos arbitrarios del sistema operativo en lugar de (o junto con) el código de visualización legítimo.

Atributo	Detalle
Identificador	CVE-2024-5565 (CWE-94, Improper Control of Generation of Code)
Severidad	CVSS 8.1 (alta); divulgada por JFrog en 2024
Punto de la Figura 1 explotado	Paso 3 → rama "Shell / API / agente": la salida del LLM (código Python de visualización) se reenvía a exec() sin validación de esquema ni sandboxing
Categoría OWASP	LLM05 (Improper Output Handling), combinada con LLM01 (Prompt Injection) y LLM08 (Excessive Agency)
Mitigación aplicable	Sandboxing del código generado, allow-list de funciones permitidas para graficar, y uso de una cuenta de base de datos de solo lectura (principio de mínimo privilegio) recomendado por el propio mantenedor de la librería

5.2 Caso 2 — CVE-2025-68664 ("LangGrinch"): fuga de secretos y RCE en LangChain

LangChain es uno de los frameworks más usados para construir agentes y aplicaciones basadas en LLM. La vulnerabilidad, bautizada "LangGrinch" por los investigadores de Cyata Security, afecta a las funciones internas de serialización y deserialización de langchain-core (dumps()/dumpd() y load()/loads()).

El problema surge porque estas funciones no escapaban correctamente los diccionarios que contuvieran la clave interna "lc", utilizada por LangChain para marcar objetos serializados propios. Como esa clave puede aparecer dentro de campos influenciados por la salida del LLM (por ejemplo, additional_kwargs o response_metadata, que se completan a partir de las respuestas del modelo durante el streaming, el registro de logs o el cacheo), un atacante que logre manipular al modelo mediante prompt injection puede lograr que datos arbitrarios se interpreten como un objeto legítimo de LangChain al momento de deserializarse. Dependiendo de la configuración, esto permitía exfiltrar secretos (credenciales de nube, cadenas de conexión, llaves de API) desde variables de entorno, e incluso derivar en ejecución remota de código.

Atributo	Detalle
Identificador	CVE-2025-68664 (Python) / CVE-2025-68665 (JavaScript); CWE502, Deserialización de datos no confiables

Severidad	CVSS 9.3 (crítica); divulgada por Cyata Security en diciembre de 2025
Punto de la Figura 1 explotado	Paso 3 → reenvío de la salida del LLM (metadatos de la respuesta) a un componente interno de serialización/deserialización, sin tratarla como dato no confiable
Categoría OWASP	LLM05 (Improper Output Handling) combinada con LLM01 (Prompt Injection) y LLM06 (Sensitive Information Disclosure)
Mitigación aplicable	Actualizar a las versiones parcheadas (0.3.81 / 1.2.5), deshabilitar la resolución automática de secretos desde variables de entorno, y auditar cualquier flujo que serialice/deserialice contenido proveniente de la salida del modelo

5.3 Lectura conjunta de los dos casos

Aunque los dos incidentes afectan componentes distintos del ecosistema de agentes de IA (una librería de texto a SQL en un caso, un framework de orquestación de agentes en el otro), ambos confirman empíricamente el argumento central de este informe: la causa raíz no está en el modelo de lenguaje, sino en la ausencia de un límite de confianza explícito entre lo que el LLM produce y lo que la aplicación ejecuta o deserializa. En ambos casos, la mitigación recomendada por los propios equipos afectados coincide con los controles descritos en la sección 6: tratar la salida del LLM como una entrada no confiable.

6. Impacto y riesgos asociados

El manejo inseguro de salidas compromete, en distintos grados, los tres pilares clásicos de la seguridad de la información:

- **Confidencialidad:** exfiltración de datos sensibles mediante XSS (robo de sesión, tokens o cookies) o mediante consultas maliciosas a bases de datos, como se observó en la fuga de secretos del caso LangGrinch (CVE-2025-68664).
- **Integridad:** modificación no autorizada de registros, archivos o configuraciones cuando la salida del LLM se ejecuta como comando o consulta con permisos de escritura.
- **Disponibilidad:** eliminación de datos, saturación de recursos o interrupción de servicios como consecuencia de comandos destructivos generados o reproducidos por el modelo, riesgo evidenciado por la capacidad de ejecución remota de código del caso Vanna.AI (CVE-2024-5565).

A nivel organizacional, la explotación de esta vulnerabilidad puede derivar en incidentes de seguridad con consecuencias reputacionales, legales y económicas, especialmente en sectores regulados donde la protección de datos personales es obligatoria. Adicionalmente, a diferencia

de una vulnerabilidad de código tradicional, el vector de entrada (el propio modelo de lenguaje) es probabilístico y difícil de predecir por completo mediante pruebas convencionales, lo que exige controles de salida robustos independientemente del comportamiento del modelo.

7. Estrategias de mitigación: validación y codificación de salidas

La recomendación central de OWASP para esta categoría de riesgo puede resumirse en un principio simple: la salida de un LLM debe tratarse como un dato de entrada no confiable (untrusted input) proveniente de un tercero, y debe pasar por los mismos controles de validación y sanitización que se aplicarían a cualquier dato externo antes de ser utilizada por otro componente del sistema.

7.1 Codificación de salida sensible al contexto (context-aware output encoding)

La salida del modelo debe codificarse de acuerdo con el contexto en el que va a utilizarse: codificación HTML si se va a renderizar en una página web, codificación de atributos si se inserta dentro de un atributo HTML, codificación de JavaScript si se incrusta en un bloque de script, o escape de URL si se utiliza como parte de un enlace. Esta práctica neutraliza el marcado potencialmente peligroso sin necesidad de interpretar o eliminar el contenido.

7.2 Validación estructurada de la salida (schema / structured output)

Cuando la respuesta del modelo debe cumplir un formato específico (JSON, un comando predefinido o parámetros de una función), es preferible forzar al modelo a producir una salida estructurada validable contra un esquema (por ejemplo, JSON Schema), y rechazar o solicitar una nueva generación si la respuesta no cumple el formato esperado. Esto reduce drásticamente la superficie de ataque, ya que limita el espacio de valores posibles antes de que la salida llegue al componente consumidor.

7.3 Consultas parametrizadas y principio de mínimo privilegio en bases de datos

En integraciones de tipo texto a SQL, la aplicación nunca debe concatenar directamente la consulta generada por el LLM. En su lugar, debe traducirse a sentencias preparadas (prepared statements) o a un constructor de consultas parametrizado, y la cuenta de base de datos utilizada por el agente debe operar bajo el principio de mínimo privilegio, con permisos de solo lectura cuando la tarea no requiera escritura.

7.4 Confinamiento (sandboxing) y listas de permitidos para ejecución de acciones

Cuando un agente ejecuta comandos, código o llamadas a herramientas a partir de la salida del modelo, dicha ejecución debe ocurrir dentro de un entorno confinado (sandbox), con acceso restringido al sistema de archivos y a la red, y limitada a un conjunto explícito de operaciones

permitidas (allow-list de herramientas o comandos), en lugar de intentar bloquear patrones maliciosos conocidos (deny-list), que son más fáciles de eludir.

7.5 Cabeceras de seguridad complementarias

Como defensa en profundidad, se recomienda complementar la sanitización de salida con una Content Security Policy (CSP) estricta en el lado del cliente, que restrinja las fuentes desde las cuales puede ejecutarse script, reduciendo el impacto de cualquier fragmento malicioso que eventualmente logre evadir la capa de sanitización.

7.6 Síntesis de controles

Vector de ataque	Control principal	Control complementario
XSS (renderizado en el DOM)	Sanitización de HTML con allow-list (p. ej. DOMPurify)	Content Security Policy (CSP)
Inyección de consultas	Sentencias preparadas / consultas parametrizadas	Mínimo privilegio en la cuenta de BD
Inyección de comandos	Sandboxing y allow-list de herramientas	Validación de esquema de salida
Reenvío a otros sistemas/agentes	Validación estructurada (JSON Schema)	Segregación de contenido no confiable en el prompt

8. Conclusiones

1. El manejo inseguro de salidas del LLM no es una falla del modelo de lenguaje en sí, sino de la arquitectura de la aplicación que lo integra: surge cuando se omite un límite de confianza claro entre lo que el modelo genera y lo que otros componentes del sistema consumen.
2. Los vectores de ataque analizados —XSS, inyección de consultas e inyección de comandos— comparten una misma causa raíz y, por lo tanto, comparten también un mismo principio de mitigación: tratar la salida del LLM como un dato externo no confiable.
3. Los casos de CVE-2024-5565 (Vanna.AI) y CVE-2025-68664 (LangChain) demuestran que esta vulnerabilidad ya ha sido explotada en la práctica sobre librerías ampliamente adoptadas, y no constituye únicamente un riesgo teórico: en ambos casos, la falta de un límite de confianza entre la salida del LLM y el componente que la consumía derivó en ejecución de código o fuga de secretos.
4. La combinación de codificación de salida sensible al contexto, validación estructurada, consultas parametrizadas, confinamiento de acciones y controles complementarios como CSP constituye una estrategia de defensa en profundidad efectiva frente a esta vulnerabilidad.

5. A medida que los agentes de IA adquieren mayor autonomía para actuar sobre otros sistemas, el manejo inseguro de salidas se vuelve más crítico, ya que se combina con el riesgo de autonomía excesiva (Excessive Agency) para amplificar el impacto potencial de un ataque exitoso.

9. Referencias

OWASP Foundation. (2025). OWASP Top 10 for LLM Applications 2025. OWASP GenAI Security Project.

Aembit. (2026). OWASP Top 10 LLM Risks Explained. Recuperado de <https://aembit.io/blog/owasp-top-10-llm-risks-explained/>

Indusface. (2026). OWASP Top 10 For LLM Applications 2025. Recuperado de <https://www.indusface.com/learning/owasp-top-10-llm/>

JFrog. (2024). When Prompts Go Rogue: Analyzing a Prompt Injection Code Execution in Vanna.AI (CVE-2024-5565). Recuperado de <https://jfrog.com/blog/prompt-injection-attackcode-execution-in-vanna-ai-cve-2024-5565/>

The Stack. (2024). CVSS 10 bug in LLM-to-SQL library makes prompt injection fun. Recuperado de <https://www.thestack.technology/cvss-10-bug-in-llm-to-sql-library-makes-prompt-injectionfun/>

CVE Program / GCVE. (2024). CVE-2024-5565 — Prompt Injection in "ask" API with visualization leads to RCE. Recuperado de <https://db.gcve.eu/vuln/cve-2024-5565>

Cyata Security. (2025). All I Want for Christmas is Your Secrets: LangGrinch hits LangChain Core (CVE-2025-68664). Recuperado de <https://cyata.ai/blog/langgrinch-langchain-core-cve-202568664/>

The Hacker News. (2025). Critical LangChain Core Vulnerability Exposes Secrets via Serialization Injection. Recuperado de <https://thehackernews.com/2025/12/critical-langchain-corevulnerability.html>

Rescana. (2025). LangGrinch (CVE-2025-68664): Critical langchain-core Vulnerability Enables Secret Exfiltration and Code Execution via Serialization Injection. Recuperado de <https://www.rescana.com/post/langgrinch-cve-2025-68664-critical-langchain-corevulnerability-enables-secret-exfiltration-and-c>